

INFORME DE FORMULACIÓN DE PROMPTS PARA LA INTEGRACIÓN DE DATOS

Evidencia: AA2-EV01

Aprendiz: Diego Alejandro Arboleda Cuero

Programa: Aplicación de la Inteligencia Artificial en la Integración de Datos

Código de Programa: 21720207

Centro de Formación: Servicio Nacional de Aprendizaje (SENA)

3 de septiembre de 2026

Ficha Técnica del Proceso Formativo

- **Competencia:** 220501115 - Integrar datos según procedimiento técnico y especificaciones de proceso de negocio[cite: 3].
 - **Resultado de Aprendizaje:** 220501115-02 - Procesar datos implementando herramientas de inteligencia artificial generativa, procedimiento técnico y metodología estadística[cite: 3].
 - **Evidencia de Aprendizaje:** AA2-EV01. Informe de formulación de prompts[cite: 3].
-

1. Introducción

El presente informe documenta el diseño, ejecución y análisis crítico de tres *prompts* estructurados bajo técnicas avanzadas de *prompt engineering* (definición de rol, contexto, tarea específica, delimitación de formato y restricciones)[cite: 1, 3]. Cada instrucción fue probada en el modelo de Inteligencia Artificial Generativa DeepSeek con el objetivo de evaluar su capacidad para automatizar tareas operativas y técnicas en flujos de integración de datos: limpieza y estandarización de registros, desarrollo de tuberías ETL para fuentes heterogéneas y generación de documentación técnica formal[cite: 1, 3].

2. Desarrollo y Evaluación de Prompts

2.1. Prompt 1: Limpieza, Imputación y Estandarización de Datos

2.1.1. Estructura del Prompt

- **Rol:** Actúa como un Ingeniero de Datos Senior especializado en calidad de datos y Python[cite: 1].
- **Contexto:** Tengo un conjunto de datos en formato CSV con registros de clientes que contiene inconsistencias en las fechas, registros duplicados y valores nulos en el campo de ingresos[cite: 1, 3].
- **Tarea:** Escribe un script en Python utilizando la biblioteca **pandas** que realice la limpieza y preparación automatizada del conjunto de datos[cite: 3].
- **Dataset de entrada:**

```
id_cliente,nombre,fecha_registro,ingreso_mensual,ciudad
101,JUAN PEREZ,2024/01/15,2500000,Bogota
102,maria gomez,15-01-2024,,Medellin
101,JUAN PEREZ,2024/01/15,2500000,Bogota
103,CARLOS RUIZ,2024-02-01,1800000,BOGOTA
```

- **Restricciones y formato:** Eliminar filas duplicadas exactas[cite: 1]. Imputar nulos en `ingreso_mensual` con la mediana según la ciudad[cite: 1]. Estandarizar `fecha_registro` al formato ISO YYYY-MM-DD[cite: 1]. Aplicar Title Case a `nombre` y `ciudad`[cite: 1]. Entregar código ejecutable y comentado[cite: 1].

2.1.2. Código Generado por la IA

```
1 import pandas as pd
2 import numpy as np
3 from io import StringIO
4
5 data_csv = """id_cliente,nombre,fecha_registro,ingreso_mensual,ciudad
6 101,JUAN PEREZ,2024/01/15,2500000,Bogota
7 102,maria gomez,15-01-2024,,Medellin
8 101,JUAN PEREZ,2024/01/15,2500000,Bogota
9 103,CARLOS RUIZ,2024-02-01,1800000,BOGOTA"""
10
11 df = pd.read_csv(StringIO(data_csv))
12 df_sin_duplicados = df.drop_duplicates(keep='first').copy()
13
14 def estandarizar_fecha(fecha_str):
15     if pd.isna(fecha_str): return np.nan
16     formatos = ['%Y/%m/%d', '%d-%m-%Y', '%Y-%m-%d']
17     for fmt in formatos:
18         try:
19             return pd.to_datetime(str(fecha_str).strip(), format=fmt).
20                 strftime('%Y-%m-%d')
21         except ValueError:
22             continue
23     try:
```

```

23         return pd.to_datetime(fecha_str).strftime('%Y-%m-%d')
24     except:
25         return np.nan
26
27 df_sin_duplicados['fecha_registro'] = df_sin_duplicados['fecha_registro']
28     ].apply(estandarizar_fecha)
29
30 mediana_por_ciudad = df_sin_duplicados.groupby('ciudad')['
31     ingreso_mensual'].median()
32
33 def imputar_ingreso(row):
34     if pd.isna(row['ingreso_mensual']):
35         ciudad = row['ciudad']
36         return mediana_por_ciudad.get(ciudad, np.nan)
37     return row['ingreso_mensual']
38
39 df_sin_duplicados['ingreso_mensual'] = df_sin_duplicados.apply(
40     imputar_ingreso, axis=1)
41 df_sin_duplicados['nombre'] = df_sin_duplicados['nombre'].astype(str).
42     str.title()
43 df_sin_duplicados['ciudad'] = df_sin_duplicados['ciudad'].astype(str).
44     str.title()

```

2.1.3. Análisis Crítico e Iteraciones

- **Evaluación de utilidad:** Alta[cite: 1]. El script resuelve de forma modular las tareas requeridas utilizando buenas prácticas en pandas[cite: 1].
- **Ajuste identificado:** En el dataset de prueba, la ciudad Medellín solo contaba con un registro y este tenía el campo ingreso_mensual nulo[cite: 1]. Al calcular la mediana por grupo, la resultante para Medellín fue NaN, por lo que el valor faltante no se imputó[cite: 1].
- **Iteración del Prompt:** Se ajustó la instrucción agregando una regla de contingencia: *“Si la mediana por ciudad resulta ser nula debido a la ausencia de datos históricos en esa ubicación, imputar con la mediana global del conjunto de datos”*[cite: 1].

2.2. Prompt 2: Pipeline ETL para Fuentes Heterogéneas (CSV y JSON)

2.2.1. Estructura del Prompt

- **Rol:** Actúa como un Especialista en Integración de Datos e Infraestructura de Información[cite: 1].
- **Contexto:** Se requiere unificar los registros de transacciones físicas (CSV) con las transacciones digitales (JSON)[cite: 3].
- **Tarea:** Genera una solución en Python que lea ambos formatos, unifique las estructuras de datos en un único DataFrame consolidado y cree una columna derivativa[cite: 1, 3].

- **Restricciones y formato:** Mapear e igualar las columnas (`id_venta` $\rightarrow$ `transaction_id`, `id_cliente` $\rightarrow$ `client_id`, `monto_total` $\rightarrow$ `total`)[cite: 1]. Crear la columna calculada `monto_con_iva` (`monto_total` $\times$ 1.19)[cite: 1]. Incluir manejo de excepciones para archivos vacíos o dañados y reporte vía `logging`[cite: 1].

2.2.2. Resultado Generado por la IA

El modelo construyó una clase orientada a objetos denominada `PipelineETL` compuesta por los métodos `leer_csv()`, `leer_json()`, `estandarizar_columnas()`, `unificar_datos()`, `calcular_iva()` y `ejecutar_pipeline()`[cite: 1]. Capturó excepciones como `FileNotFoundError` y `JSONDecodeError`, e incorporó un script de prueba que validó el cálculo del 19% de IVA y la deduplicación de transacciones[cite: 1].

2.2.3. Análisis Crítico e Iteraciones

- **Evaluación de utilidad:** Muy alta[cite: 1]. La arquitectura generada presenta un nivel profesional con trazabilidad mediante `logging`[cite: 1].
- **Ajuste identificado:** Para integraciones ad-hoc o análisis rápidos, la estructura basada en clases completas puede resultar sobre-dimensionada[cite: 1].
- **Iteración del Prompt:** Se refinó la instrucción para solicitar opcionalmente: *“Proporcionar además una versión funcional simplificada mediante funciones puras para procesamiento rápido”*[cite: 1].

2.3. Prompt 3: Documentación Técnica y Diccionario de Datos

2.3.1. Estructura del Prompt

- **Rol:** Actúa como un Arquitecto de Datos y Analista de Gobierno de Datos[cite: 1].
- **Contexto:** Se ha construido una tabla consolidada denominada `fact_ventas_integradas` que unifica los canales físicos y digitales[cite: 3].
- **Tarea:** Diseña la documentación técnica formal y el diccionario de datos para esta estructura[cite: 3].
- **Restricciones y formato:** Incluir descripción conceptual orientada al equipo de analítica[cite: 1, 3]. Presentar el diccionario en una tabla Markdown con las columnas: Nombre del Campo, Tipo de Dato SQL, Permite Nulos, Descripción Técnica y Regla de Negocio Asociada[cite: 1].

2.3.2. Resultado Generado por la IA

Descripción General del Modelo de Datos La tabla `fact_ventas_integradas` constituye el núcleo transaccional unificado del negocio, consolidando información de ventas provenientes tanto de canales físicos como digitales[cite: 3]. Este modelo ha sido diseñado bajo esquema en estrella para responder consultas analíticas con consistencia y alto rendimiento[cite: 1]. Ofrece al equipo de analítica una vista depurada que elimina los silos de información, permitiendo evaluar la efectividad omnicanal y patrones de consumo[cite: 1, 3]. La granularidad corresponde a la transacción individual y todas las métricas monetarias incluyen el cálculo homogéneo de impuestos[cite: 1].

Campo	Tipo SQL	Nulos	Descripción Técnica	Regla de Negocio
id_venta	BIGINT	No	Identificador único universal de la transacción (PK)[cite: 1].	Trazabilidad unívoca. Sin duplicados ni reinicios de secuencia[cite: 1].
id_cliente	BIGINT	Sí	Clave foránea referenciando a <code>dim_cliente</code> (FK)[cite: 1].	Permite NULL para ventas de clientes invitados o sin registro[cite: 1].
monto_total	DECIMAL(18,2)	No	Monto bruto antes de impuestos en moneda base[cite: 1].	Debe ser ≥ 0 . Devoluciones se gestionan en tablas separadas[cite: 1].
monto_con_iva	DECIMAL(18,2)	No	Monto final con Impuesto al Valor Agregado (IVA)[cite: 1].	Calculado como $\text{monto_total} \times (1 + \text{tasa_iva})$. Siempre $\geq \text{monto_total}$ [cite: 1].
canal_origen	VARCHAR(50)	No	Código del punto de contacto comercial[cite: 1].	Valores permitidos: <code>FISICO_TIENDA</code> , <code>DIGITAL_WEB</code> , <code>DIGITAL_APP</code> [cite: 1].

Cuadro 1: Diccionario de datos de la estructura `fact_ventas_integradas`.

Diccionario de Datos – `fact_ventas_integradas`

2.3.3. Análisis Crítico e Iteraciones

- **Evaluación de utilidad:** Sobresaliente[cite: 1]. La incorporación de reglas de negocio específicas (como el manejo explícito de compras anónimas mediante valores NULL) añade rigor al gobierno de datos[cite: 1].
- **Iteración del Prompt:** No requirió modificaciones sustanciales de fondo, ya que el modelo cumplió con todas las restricciones impuestas[cite: 1].

3. Conclusiones

1. **Eficiencia Operativa:** La aplicación de técnicas estructuradas de *prompt engineering* permite acelerar drásticamente los tiempos de desarrollo de scripts de integración de datos y documentación técnica[cite: 1, 3].
2. **Relevancia de las Restricciones:** Definir reglas de negocio claras en la instrucción evita inconsistencias lógicas en el código producido por modelos generativos[cite: 1].
3. **Supervisión Técnica:** Aunque la IA es altamente efectiva para generar código y documentación, el criterio humano del analista o ingeniero de datos sigue siendo indispensable para auditar casos borde e imprevistos en la calidad del dato[cite: 1, 3].